UTN
Facultad Regional Córdoba

TUP

Tecnicatura Universitaria en Programación

LABORATORIO DE COMPUTACIÓN I

Unidad Temática N°4:
Consultas Continuación

Guía Práctica
1° Año – 1° Cuatrimestre

CC BY ND NC

Índice

Problema N°1: Funciones de agregado y Consultas Sumarias 2
Problema N°2: Consultas agrupadas: Cláusula GROUP BY 4
Problema N°3: Procedimientos Almacenados 8
Solución del Problema N°1: Funciones de agregado y Consultas Sumarias 9
Solución del Problema N°2: Consultas agrupadas: Cláusula GROUP BY 11
Solución del Problema N°3: Procedimientos Almacenados 13
BIBLIOGRAFÍA **17**

Problema N°1: Funciones de agregado y Consultas Sumarias

Para llevar un mejor control del funcionamiento del negocio de la base de datos "Librería", se pretende obtener datos totalizadores, como, por ejemplo:

1.1 Facturación total del negocio

Para poder obtener este tipo de resultados, se utilizarán consultas sumarias que emplean un conjunto de funciones de columnas o **funciones de agregado**.

En este caso particular, se utilizará la función **SUM** para realizar la sumatoria de la cantidad por el precio unitario de todos los registros de la tabla detalle de facturas.

La solución sería la siguiente:

```
SELECT SUM(pre_unitario * cantidad) 'FACTURACION TOTAL'
  FROM detalle_facturas;
```

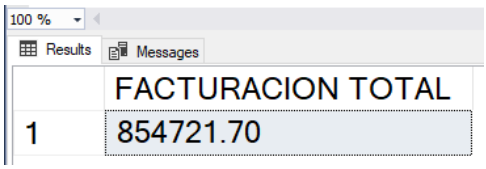

Imagen 1: Elaboración propia.

1.2 También se quiere saber el total de la factura Nro. 34, la cantidad de artículos vendidos, cantidad de ventas, el precio máximo y mínimo vendido.

Para lograr esto, se utilizarán diferentes funciones en la consulta SQL. Primero, se calculará el importe total multiplicando el precio unitario de cada artículo por la cantidad correspondiente y sumando los resultados. Luego, se sumarán todas las cantidades de artículos vendidos. La función **COUNT(*)** se utilizará para contar el número total de registros en la tabla. Además, se utilizarán las funciones **MAX** y **MIN** para encontrar el precio unitario máximo y mínimo de los artículos, respectivamente. Todo esto se realizará filtrando los registros por el número de factura solicitado en el **WHERE**. La consulta SQL mejorada es la siguiente:

```
SELECT SUM(DF.PRE_UNITARIO * DF.CANTIDAD) 'IMPORTE TOTAL',
       SUM(DF.CANTIDAD) 'CANTIDAD ARTICULOS',
       COUNT(*) 'CANTIDAD VENTAS',
       MAX(DF.PRE_UNITARIO) 'PRECIO MAXIMO',
       MIN(DF.PRE_UNITARIO) 'PRECIO MINIMO'
  FROM DETALLE_FACTURAS DF
 WHERE DF.NRO_FACTURA = 34;
```

	IMPORTE TOTAL	CANTIDAD ARTICULOS	CANTIDAD VENTAS	PRECIO MAXIMO	PRECIO MINIMO
1	645.00	60	5	101.00	1.00

Imagen 2: Elaboración propia.

1.3 Se nos solicita además lo siguiente: ¿Cuánto se facturó el año pasado?

En este caso es necesaria la tabla factura ya que en ella se encuentra la fecha.

```
SELECT SUM(PRE_UNITARIO * CANTIDAD)'FACTURACIÓN TOTAL DEL AÑO ANTERIOR'
  FROM DETALLE_FACTURAS D, FACTURAS F
 WHERE D.NRO_FACTURA = F.NRO_FACTURA
   AND YEAR(FECHA) = YEAR(GETDATE())- 1;
```

Hay que tener presente que la lista de selección **solo puede** contener campos o expresiones dentro de una función de agregado (o función de columna).

Resuelva el resto de los requerimientos del **problema 1** de los usuarios del sistema:

1.4 Se quiere saber la cantidad de ventas que hizo el vendedor de código 3.

1.5 Resolver la misma consulta anterior, pero no por el código sino por el nombre ‘Alejandro’ y apellido ‘Lopez’ del vendedor.

1.6 ¿Cuál fue la fecha de la primera y última venta que se realizó en este negocio?

1.7 Mostrar la siguiente información respecto a la factura Nro.: 450: cantidad total de unidades vendidas, la cantidad de artículos diferentes vendidos y el importe total.

1.8 ¿Cuál fue la cantidad total de unidades vendidas, importe total y el importe promedio para vendedores cuyos nombres comienzan con letras que van de la “d” a la “l”?

1.9 Se quiere saber el importe total vendido, el promedio del importe vendido y la cantidad total de artículos vendidos para el cliente Roque Paez.

1.10 Mostrar la fecha de la primera venta, la cantidad total vendida y el importe total vendido para los artículos que empiecen con “C”.

1.11 Se quiere saber la cantidad total de artículos vendidos y el importe total vendido para el periodo del 15/06/2011 al 15/06/2017.

1.12 Se quiere saber la cantidad de veces y la última vez que vino el cliente de apellido Abarca y cuánto gastó en total.

1.13 Mostrar el importe total y el promedio del importe para los clientes cuya dirección de mail es conocida.

1.14 Obtener la siguiente información: el importe total vendido y el importe promedio vendido para números de factura que no sean los siguientes: 13, 5, 17, 33, 24.

Problema N°2: Consultas agrupadas: Cláusula GROUP BY

El gerente del negocio requiere otro tipo de información resumida, pero no totales generales, sino totales agrupados por algún criterio en particular. Por ejemplo:

2.1 Los importes totales de ventas por cada artículo que se encuentra en el negocio.

Para lograr esto, vamos a implementar el siguiente enfoque: Las consultas sumarias funcionan como totales finales de un informe. Si necesitamos resumir los resultados de la consulta a un nivel de subtotal, se utiliza la cláusula **GROUP BY** en la sentencia **SELECT**. ***Esto implica agrupar los registros según los valores de una o más columnas y obtener totales para cada grupo***.

Aquí está la consulta que nos proporcionará los importes totales de ventas por cada artículo en el negocio:

```
SELECT DF.COD_ARTICULO,
       SUM(DF.PRE_UNITARIO * DF.CANTIDAD) 'TOTAL VENTAS'
  FROM DETALLE_FACTURAS DF
 GROUP BY DF.COD_ARTICULO;
```

	COD_ARTICULO	TOTAL VENTAS
1	23	10123.50
2	15	2121.00
3	9	11229.70
4	3	167505.00
5	12	16567.50
6	6	46553.00
7	7	9433.55
8	1	31252.00
9	24	75.00
10	18	40899.00
11	10	2787.20

Imagen 2: Elaboración propia.

En el primer punto del problema, debemos calcular la sumatoria del precio unitario por la cantidad para cada artículo diferente y obtener el resultado. Para lograr esto, utilizamos la cláusula SELECT y agrupamos los resultados por el campo "cod_articulo" de la tabla "detalle_facturas". La **función de agregado** **SUM** se aplica a la expresión "pre_unitario * cantidad". Esto nos dará el total de ventas por artículo.

Observe la lista de selección (cláusula select):

```
SELECT DF.COD_ARTICULO,
       SUM(DF.PRE_UNITARIO * DF.CANTIDAD) 'TOTAL VENTAS'
```

Ya no solo hemos incorporado una expresión dentro de una función de agregado, sino que además hay una columna extra: **esta columna no es ni más ni menos que la misma por la que agrupamos**:

```
GROUP BY cod_articulo
```

Esto estaría dando la facturación total por artículo de absolutamente toda la venta. Otro ejemplo es si queremos limitar los resultados al año pasado y ordenarlos por el código de artículo, necesitamos agregar la tabla "facturas" y establecer la relación con la tabla "detalle_facturas" en la cláusula **WHERE**. Además, añadimos la condición de búsqueda para el año pasado utilizando la función **YEAR** y la función **GETDATE()**. La consulta sería la siguiente:

```
SELECT DF.COD_ARTICULO,
       SUM(DF.PRE_UNITARIO * DF.CANTIDAD) 'TOTAL VENTAS'
FROM FACTURAS F, DETALLE_FACTURAS DF
 WHERE F.NRO_FACTURA = DF.NRO_FACTURA
AND YEAR(F.FECHA) = YEAR(GETDATE()) - 1
GROUP BY DF.COD_ARTICULO
ORDER BY DF.COD_ARTICULO;
```

Se puede observar que la lista de selección contiene únicamente campos o expresiones dentro de una función de agregado (función sumaria) y la columna (o columnas) de agrupación es decir el campo (o campos) incluido en el GROUP BY.

	COD_ARTICULO	TOTAL POR ARTÍCULO
1	1	30294.00
2	2	72974.00
3	3	145350.00
4	4	1987.50
5	5	12434.10
6	6	31112.50
7	7	5304.50
8	8	40481.90
9	9	10670.00
10	10	2187.00

Imagen 3: Elaboración propia.

Múltiples columnas de agrupación.

SQL puede agrupar resultados de consultas en base a contenidos de dos o más columnas. Por ejemplo, calcular el total facturado por cada vendedor y a cada cliente el año pasado ordenado por vendedor primero y luego por cliente:

```
select v.cod_vendedor,ape_vendedor+' '+nom_vendedor'Vendedor', ape_cliente+' '+nom_cliente'Cliente',sum(pre_unitario*cantidad)'Total'
from  detalle_facturas d, facturas f,vendedores v, clientes c
where d.nro_factura=F.nro_factura and f.cod_cliente=c.cod_cliente
 and  f.cod_vendedor=v.cod_vendedor and year(fecha)=year(getdate())-1
group by v.cod_vendedor,ape_vendedor+' '+nom_vendedor,c.cod_cliente,ape_cliente+' '+nom_cliente
order by 2,3
```

100 %

Results | Messages

	cod_vendedor	Vendedor	Cliente	Total
1	1	Carrizo Martín	Abarca Héctor	817.00
2	1	Carrizo Martín	Castillo Marta Analía	558.00
3	1	Carrizo Martín	Luque Elvira Josefa	150.00
4	1	Carrizo Martín	Morales Santiago	3390.00
5	1	Carrizo Martín	Paez Roque	200.00
6	1	Carrizo Martín	Perez Carlos Antonio	3406.00
7	1	Carrizo Martín	Perez Rodolfo	551.50
8	1	Carrizo Martín	Ruiz Marcos	800.00
9	2	Ledesma Mariela	Abarca Héctor	996.00
10	2	Ledesma Mariela	Castillo Marta Analía	120.00
11	2	Ledesma Mariela	Luque Elvira Josefa	4759.00
12	2	Ledesma Mariela	Morales Pilar	4665.00
13	2	Ledesma Mariela	Morales Santiago	2692.50
14	2	Ledesma Mariela	Paez Roque	2690.00
15	2	Ledesma Mariela	Perez Carlos Antonio	299.00
16	2	Ledesma Mariela	Perez Rodolfo	1672.00
17	2	Ledesma Mariela	Ruiz Marcos	2574.50
18	3	Lopez Alejandro	Abarca Héctor	1174.00

Esta consulta respondería a la pregunta: ¿Cuánto le vendió cada vendedor a cada cliente el año pasado?

Observe detenidamente, qué es lo que se incluye como columnas de agrupación (en el *GROUP BY*) y qué columnas son las que se utilizaron en la lista de selección.

Ahora puede seguir resolviendo los requerimientos del problema 2:

2.2 Por cada factura emitida mostrar la cantidad total de artículos vendidos (suma de las cantidades vendidas), la cantidad ítems que tiene cada factura en el detalle (cantidad de registros de detalles) y el Importe total de la facturación del año 2022.

2.3 Se quiere saber en este negocio, cuánto se factura:

- Diariamente
- Mensualmente
- Anualmente

2.4 Emitir un listado de la cantidad de facturas confeccionadas diariamente, correspondiente a los meses que no sean enero, julio ni diciembre. Ordene por la cantidad de facturas en forma descendente y fecha.

2.5 Se quiere saber la cantidad y el importe promedio vendido por fecha y cliente, para códigos de vendedor superiores a 2. Ordene por fecha y cliente.

2.6 Se quiere saber el importe promedio vendido y la cantidad total vendida por fecha y artículo, para códigos de cliente inferior a 3. Ordene por fecha y artículo.

2.7 Listar la cantidad total vendida, el importe total vendido y el importe promedio total vendido por número de factura, siempre que la fecha no oscile entre el 13/2/2007 y el 13/7/2010.

2.8 Emitir un reporte que muestre la fecha de la primer y última venta y el importe comprado por cliente. Rotule como CLIENTE, PRIMER VENTA, ÚLTIMA VENTA, IMPORTE.

2.9 Se quiere saber el importe total vendido, la cantidad total vendida y el precio unitario promedio por cliente y artículo, siempre que el nombre del cliente comience con letras que van de la “a” a la “m”. Ordene por cliente, precio unitario promedio en forma descendente y artículo. Rotule como IMPORTE TOTAL, CANTIDAD TOTAL, PRECIO PROMEDIO.

2.10 Se quiere saber la cantidad de facturas y la fecha la primer y última factura por vendedor y cliente, para números de factura que oscilan entre 5 y 30. Ordene por vendedor, cantidad de ventas en forma descendente y cliente.

Problema N°3: Procedimientos Almacenados

3.1 Cree los siguientes **Procedimientos Almacenados** (**PA**) o **Store Procedure** (**SP**):

a. **DETALLE_VENTAS**: Crear **PA** o **SP** que liste la fecha, la factura, el vendedor, el cliente, el artículo, cantidad e importe. Este **PA** o **SP** recibirá como parámetros de entrada un rango de fechas.

b. **CANTIDAD_ART_CLI**: Crear **PA** o **SP** que me debe devolver la cantidad de artículos o clientes (según se pida) que existen en la empresa.

c. **INS_VENDEDOR**: Cree un **PA** o **SP** que le permita insertar registros en la tabla vendedores.

d. **UPD_VENDEDOR**: cree un **PA** o **SP** que le permita modificar un vendedor cargado.

e. **DEL_VENDEDOR**: cree un **PA** o **SP** que le permita eliminar un vendedor ingresado.

3.2 Modifique el **PA** o **SP** del punto **3.1 a**, permitiendo que los resultados del **PA** o **SP** puedan filtrarse por una fecha determinada, por un rango de fechas y por un rango de vendedores; según se pida.

3.3 Ejecute los **PA** o **SP** creados en el punto 3.1 (todos).

3.4 Elimine los **PA** o **SP** creados en el punto 3.1.

Solución del Problema N°1: Funciones de agregado y Consultas Sumarias

1.1

```
SELECT SUM(PRE_UNITARIO * CANTIDAD) 'FACTURACION TOTAL'
  FROM DETALLE_FACTURAS;
```

1.2

```
SELECT SUM(DF.PRE_UNITARIO * DF.CANTIDAD) 'IMPORTE TOTAL',
       SUM(DF.CANTIDAD) 'CANTIDAD ARTICULOS',
       COUNT(*) 'CANTIDAD VENTAS',
       MAX(DF.PRE_UNITARIO) 'PRECIO MAXIMO',
       MIN(DF.PRE_UNITARIO) 'PRECIO MINIMO'
  FROM DETALLE_FACTURAS DF
 WHERE DF.NRO_FACTURA = 34;
```

1.3

```
SELECT SUM(PRE_UNITARIO * CANTIDAD)'FACTURACIÓN TOTAL DEL AÑO ANTERIOR'
  FROM DETALLE_FACTURAS D, FACTURAS F
 WHERE D.NRO_FACTURA = F.NRO_FACTURA
   AND YEAR(FECHA) = YEAR(GETDATE())- 1;
```

1.4

```
SELECT COUNT(*) 'CANTIDAD DE VENTAS'
  FROM FACTURAS
 WHERE COD_VENDEDOR = 3;
```

1.5

```
SELECT COUNT(*) 'CANTIDAD DE VENTAS'
  FROM FACTURAS F, VENDEDORES V
 WHERE F.COD_VENDEDOR = V.COD_VENDEDOR
   AND V.NOM_VENDEDOR = 'Alejandro'
   AND V.APE_VENDEDOR = 'Lopez';
```

1.6

```
SELECT MIN(F.FECHA) 'FECHA PRIMERA VENTA',
       MAX(F.FECHA) 'FECHA ULTIMA VENTA'
  FROM FACTURAS F;

-- Consulta equivalente que mejora la presentación
SELECT CONVERT(VARCHAR, MIN(F.FECHA), 103) 'FECHA PRIMERA VENTA',
       CONVERT(VARCHAR, MAX(F.FECHA), 103) 'FECHA ULTIMA VENTA'
FROM FACTURAS F;
```

1.7

```
SELECT SUM(DF.CANTIDAD) 'CANT. TOTAL UNID. VENDIDAS',
       COUNT(DISTINCT DF.COD_ARTICULO) 'CANT. ARTÍCULOS VENDIDOS',
       SUM(DF.PRE_UNITARIO * DF.CANTIDAD) 'IMPORTE TOTAL'
  FROM DETALLE_FACTURAS DF
 WHERE DF.NRO_FACTURA = 450;
```

1.8

```
SELECT SUM(DF.CANTIDAD)'CANT. TOTAL UNID. VENDIDAS',
       SUM(DF.PRE_UNITARIO * DF.CANTIDAD) 'IMPORTE TOTAL',
       AVG(DF.PRE_UNITARIO * DF.CANTIDAD)'PROMEDIO IMPORTE TOTAL'
  FROM FACTURAS F, DETALLE_FACTURAS DF, VENDEDORES V
 WHERE F.NRO_FACTURA = DF.NRO_FACTURA
   AND F.COD_VENDEDOR = V.COD_VENDEDOR
   AND V.NOM_VENDEDOR LIKE '[d-l]%';
```

1.9

```
SELECT SUM(DF.PRE_UNITARIO * DF.CANTIDAD) 'IMPORTE TOTAL VENDIDO',
       AVG(DF.PRE_UNITARIO * DF.CANTIDAD) 'PROMEDIO TOTAL VENDIDO',
       SUM(DF.CANTIDAD) 'CANTIDAD TOTAL VENDIDO'
  FROM FACTURAS F, DETALLE_FACTURAS DF, CLIENTES C
 WHERE F.NRO_FACTURA = DF.NRO_FACTURA
   AND F.COD_CLIENTE = C.COD_CLIENTE
   AND C.NOM_CLIENTE = 'Roque'
   AND C.APE_CLIENTE = 'Paez';
```

1.10

```
SELECT MIN(F.FECHA) 'FECHA PRIMERA VENTA',
       SUM(DF.CANTIDAD) 'CANTIDAD TOTAL VENDIDA',
       SUM(DF.PRE_UNITARIO * DF.CANTIDAD) 'IMPORTE TOTAL VENDIDO'
  FROM FACTURAS F, DETALLE_FACTURAS DF, ARTICULOS A
 WHERE F.NRO_FACTURA = DF.NRO_FACTURA
   AND DF.COD_ARTICULO = A.COD_ARTICULO
   AND A.DESCRIPCION LIKE 'C%';
```

1.11

```
SELECT SUM(DF.CANTIDAD) 'CANTIDAD TOTAL ARTÍCULOS',
       SUM(DF.PRE_UNITARIO * DF.CANTIDAD) 'IMPORTE TOTAL VENDIDO'
  FROM FACTURAS F, DETALLE_FACTURAS DF
 WHERE F.NRO_FACTURA = DF.NRO_FACTURA
   AND F.FECHA BETWEEN '15/06/2011' AND '15/06/2017';

-- Consulta equivalente
SELECT SUM(DF.CANTIDAD) 'CANTIDAD TOTAL ARTÍCULOS',
       SUM(DF.PRE_UNITARIO * DF.CANTIDAD) 'IMPORTE TOTAL VENDIDO'
  FROM FACTURAS F, DETALLE_FACTURAS DF
 WHERE F.NRO_FACTURA = DF.NRO_FACTURA
   AND F.FECHA >= '15/06/2011' AND F.FECHA <= '15/06/2017';
```

1.12

```
SELECT COUNT(*) AS 'CANTIDAD VISITA',
       MAX(F.FECHA) 'ULITMA VISITA',
       SUM(DF.PRE_UNITARIO * DF.CANTIDAD) 'TOTAL GASTADO'
  FROM FACTURAS F, DETALLE_FACTURAS DF, CLIENTES C
 WHERE F.NRO_FACTURA = DF.NRO_FACTURA
   AND F.COD_CLIENTE = C.COD_CLIENTE
   AND C.APE_CLIENTE = 'Abarca';
```

1.13

```
SELECT SUM(DF.PRE_UNITARIO * DF.CANTIDAD) 'IMPORTE TOTAL',
       AVG(DF.PRE_UNITARIO * DF.CANTIDAD) 'PROMEDIO DEL IMPORTE '
  FROM FACTURAS F, DETALLE_FACTURAS DF, CLIENTES C
 WHERE F.NRO_FACTURA = DF.NRO_FACTURA
   AND F.COD_CLIENTE = C.COD_CLIENTE
   AND C.[E-MAIL] IS NOT NULL;
```

1.14

```
SELECT SUM(DF.PRE_UNITARIO * DF.CANTIDAD) 'IMPORTE TOTAL VENDIDO',
       AVG(DF.PRE_UNITARIO * DF.CANTIDAD) 'IMPORTE PROMEDIO VENDIDO'
  FROM FACTURAS F, DETALLE_FACTURAS DF
 WHERE F.NRO_FACTURA = DF.NRO_FACTURA
   AND F.NRO_FACTURA NOT IN (13, 5, 17, 33, 24);
```

Solución del Problema N°2: Consultas agrupadas: Cláusula GROUP BY

2.2

```
SELECT F.NRO_FACTURA,
       SUM(DF.CANTIDAD) 'CANTIDAD VENDIDAS',
       COUNT(*) 'CANTIDAD ITEMS',
       SUM(DF.PRE_UNITARIO * DF.CANTIDAD) 'IMPORTE TOTAL'
  FROM FACTURAS F, DETALLE_FACTURAS DF
 WHERE F.NRO_FACTURA = DF.NRO_FACTURA
   AND YEAR(F.FECHA) = 2022
 GROUP BY F.NRO_FACTURA
 ORDER BY F.NRO_FACTURA;
```

2.3

```
-- (DIARIAMENTE) SUMA EL IMPORTE AGRUPADO POR DÍA DE TODOS LOS AÑOS
SELECT DAY(F.FECHA) 'DIA',
       SUM(DF.PRE_UNITARIO * DF.CANTIDAD) 'IMPORTE TOTAL'
  FROM FACTURAS F, DETALLE_FACTURAS DF
 WHERE F.NRO_FACTURA = DF.NRO_FACTURA
 GROUP BY DAY(F.FECHA)
 ORDER BY 1;

-- (MENSUALMENTE) SUMA EL IMPORTE AGRUPADO POR MES DE TODOS LOS AÑOS
SELECT MONTH(F.FECHA) 'MES',
       SUM(DF.PRE_UNITARIO * DF.CANTIDAD) 'IMPORTE TOTAL'
  FROM FACTURAS F, DETALLE_FACTURAS DF
 WHERE F.NRO_FACTURA = DF.NRO_FACTURA
 GROUP BY MONTH(F.FECHA)
 ORDER BY 1;

-- (ANUALMENTE) SUMA EL IMPORTE AGRUPADO POR AÑO
SELECT YEAR(F.FECHA) 'AÑO',
       SUM(DF.PRE_UNITARIO * DF.CANTIDAD) 'IMPORTE TOTAL'
  FROM FACTURAS F, DETALLE_FACTURAS DF
 WHERE F.NRO_FACTURA = DF.NRO_FACTURA
 GROUP BY YEAR(F.FECHA)
 ORDER BY 1;
```

2.4

```
SELECT DAY(F.FECHA) 'DIA',
       COUNT(F.NRO_FACTURA) 'CANT. FACTURAS'
  FROM FACTURAS F, DETALLE_FACTURAS DF
 WHERE F.NRO_FACTURA = DF.NRO_FACTURA
   AND MONTH(F.FECHA) NOT IN (1, 7, 12)
 GROUP BY DAY(F.FECHA)
 ORDER BY 1;
```

2.5

```
SELECT F.FECHA,
       C.COD_CLIENTE,
       COUNT(DF.COD_ARTICULO) 'CANTIDAD',
       AVG(DF.PRE_UNITARIO * DF.CANTIDAD) 'IMPORTE PROMEDIO'
  FROM FACTURAS F
  JOIN DETALLE_FACTURAS DF ON F.NRO_FACTURA = DF.NRO_FACTURA
  JOIN CLIENTES C ON F.COD_CLIENTE = C.COD_CLIENTE
  JOIN VENDEDORES V ON F.COD_VENDEDOR = V.COD_VENDEDOR
 WHERE V.COD_VENDEDOR > 2
 GROUP BY F.FECHA, C.COD_CLIENTE
 ORDER BY F.FECHA, C.COD_CLIENTE;
```

```
-- UNA FORMA PARA VISUALIZAR MEJOR LA FECHA:
-- CONVERT(VARCHAR, F.Fecha, 103) 'FECHA',
```

2.6

```
SELECT F.FECHA,
       A.COD_ARTICULO,
       AVG(DF.PRE_UNITARIO * DF.CANTIDAD) 'IMPORTE PROMEDIO',
       SUM(DF.CANTIDAD) 'CANTIDAD TOTAL'
  FROM FACTURAS F
  JOIN DETALLE_FACTURAS DF ON F.NRO_FACTURA = DF.NRO_FACTURA
  JOIN ARTICULOS A ON DF.COD_ARTICULO = A.COD_ARTICULO
  JOIN CLIENTES C ON F.COD_CLIENTE = C.COD_CLIENTE
 WHERE C.COD_CLIENTE < 3
 GROUP BY F.FECHA, A.COD_ARTICULO
 ORDER BY F.FECHA, A.COD_ARTICULO;

-- UNA FORMA PARA VISUALIZAR MEJOR LA FECHA:
-- CONVERT(VARCHAR, F.Fecha, 103) 'FECHA',
```

2.7

```
SELECT F.NRO_FACTURA 'NRO. FACTURA',
       SUM(DF.CANTIDAD) 'CANTIDAD TOTAL',
       SUM(DF.PRE_UNITARIO * DF.CANTIDAD) 'IMPORTE TOTAL',
       AVG(DF.PRE_UNITARIO * DF.CANTIDAD) 'IMPORTE PROMEDIO'
  FROM FACTURAS F
  JOIN DETALLE_FACTURAS DF ON F.NRO_FACTURA = DF.NRO_FACTURA
 WHERE F.FECHA NOT BETWEEN '13/02/2007' AND '13/07/2010'
 GROUP BY F.NRO_FACTURA
```

2.8

```
SELECT C.COD_CLIENTE 'CLIENTE',
       MIN(F.FECHA) 'PRIMER VENTA',
       MAX(F.FECHA) 'ULTIMA VENTA',
       SUM(DF.PRE_UNITARIO * DF.CANTIDAD) 'IMPORTE'
  FROM CLIENTES C
  JOIN FACTURAS F ON C.COD_CLIENTE = F.COD_CLIENTE
  JOIN DETALLE_FACTURAS DF ON F.NRO_FACTURA = DF.NRO_FACTURA
 GROUP BY C.COD_CLIENTE

-- UNA FORMA PARA VISUALIZAR MEJOR LA FECHA:
-- CONVERT(VARCHAR, MIN(F.FECHA), 103) 'PRIMER VENTA',
-- CONVERT(VARCHAR, MAX(F.FECHA), 103) 'ULTIMA VENTA',
```

2.9

```
SELECT C.NOM_CLIENTE 'CLIENTE',
       A.COD_ARTICULO 'ARTICULO',
       SUM(DF.PRE_UNITARIO * DF.CANTIDAD) 'IMPORTE TOTAL',
       SUM(DF.CANTIDAD) 'CANTIDAD TOTAL',
       AVG(DF.PRE_UNITARIO) 'PRECIO PROMEDIO'
  FROM CLIENTES C
  JOIN FACTURAS F ON C.COD_CLIENTE = F.COD_CLIENTE
  JOIN DETALLE_FACTURAS DF ON F.NRO_FACTURA = DF.NRO_FACTURA
  JOIN ARTICULOS A ON DF.COD_ARTICULO = A.COD_ARTICULO
 WHERE C.NOM_CLIENTE LIKE '[A-M]%'
 GROUP BY C.NOM_CLIENTE, A.COD_ARTICULO
 ORDER BY C.NOM_CLIENTE, 'PRECIO PROMEDIO' DESC, A.COD_ARTICULO;
```

Solución del Problema N°3: Procedimientos Almacenados

3.1

```
-- (A)

CREATE PROCEDURE DETALLE_VENTAS
    @FECHAINICIO DATETIME,
    @FECHAFIN DATETIME
AS
BEGIN
    SELECT F.FECHA 'FECHA',
           F.NRO_FACTURA 'FACTURA',
           V.NOM_VENDEDOR, ' ', V.APE_VENDEDOR 'VENDEDOR',
           C.NOM_CLIENTE, ' ', C.APE_CLIENTE 'CLIENTE',
           A.DESCRIPCION 'ARTICULO',
           DF.CANTIDAD 'CANTIDAD',
           (DF.PRE_UNITARIO * DF.CANTIDAD) 'IMPORTE'
      FROM FACTURAS F
      JOIN VENDEDORES V ON F.COD_VENDEDOR = V.COD_VENDEDOR
      JOIN CLIENTES C ON F.COD_CLIENTE = C.COD_CLIENTE
      JOIN DETALLE_FACTURAS DF ON F.NRO_FACTURA = DF.NRO_FACTURA
      JOIN ARTICULOS A ON DF.COD_ARTICULO = A.COD_ARTICULO
     WHERE F.FECHA BETWEEN @FECHAINICIO AND @FECHAFIN
     ORDER BY F.FECHA, F.NRO_FACTURA
END;



-- (B)

CREATE PROCEDURE CANTIDAD_ART_CLI
    @TIPOCONSULTA VARCHAR(10)
     -- "ARTICULOS" PARA CONTAR LA CANTIDAD DE ARTÍCULOS
     -- "CLIENTES" PARA CONTAR LA CANTIDAD DE CLIENTES
AS
BEGIN

    IF @TIPOCONSULTA = 'ARTICULOS'

        BEGIN
            SELECT COUNT(*) AS CANTIDADARTICULOS
              FROM ARTICULOS
        END

    ELSE IF @TIPOCONSULTA = 'CLIENTES'

        BEGIN
            SELECT COUNT(*) AS CANTIDADCLIENTES
            FROM CLIENTES
        END

END;
```

```sql
-- (C)

CREATE PROCEDURE INS_VENDEDOR
    @NOM_VENDEDOR NVARCHAR(50),
    @APE_VENDEDOR NVARCHAR(50),
    @CALLE        NVARCHAR(50),
    @ALTURA       INT,
    @COD_BARRIO   INT,
    @NRO_TEL      BIGINT,
    @EMAIL        NVARCHAR(50),
    @FEC_NAC      DATETIME
AS
BEGIN
    INSERT INTO VENDEDORES (
        NOM_VENDEDOR,
        APE_VENDEDOR,
        CALLE,
        ALTURA,
        COD_BARRIO,
        NRO_TEL,
        [E-MAIL],
        FEC_NAC)
    VALUES (
        @NOM_VENDEDOR,
        @APE_VENDEDOR,
        @CALLE,
        @ALTURA,
        @COD_BARRIO,
        @NRO_TEL,
        @EMAIL,
        @FEC_NAC);
END;


-- (D)

CREATE PROCEDURE UPD_VENDEDOR
    @COD_VENDEDOR INT,
    @NOM_VENDEDOR NVARCHAR(50),
    @APE_VENDEDOR NVARCHAR(50),
    @CALLE        NVARCHAR(50),
    @ALTURA       INT,
    @COD_BARRIO   INT,
    @NRO_TEL      BIGINT,
    @EMAIL        NVARCHAR(50),
    @FEC_NAC      DATETIME
AS
BEGIN

    UPDATE VENDEDORES
       SET NOM_VENDEDOR = @NOM_VENDEDOR,
           APE_VENDEDOR = @APE_VENDEDOR,
           CALLE = @CALLE,
           ALTURA = @ALTURA,
           COD_BARRIO = @COD_BARRIO,
           NRO_TEL = @NRO_TEL,
           [E-MAIL] = @EMAIL,
           FEC_NAC = @FEC_NAC
     WHERE COD_VENDEDOR = @COD_VENDEDOR;

END;
```

```
-- (E)
CREATE PROCEDURE DEL_VENDEDOR
    @cod_vendedor int
AS
BEGIN
    DELETE FROM vendedores
     WHERE cod_vendedor = @cod_vendedor
END;
```

3.3

```
-- (A)
-- UN EJEMPLO DE EJECUCIÓN
EXEC DETALLE_VENTAS @FECHAINICIO = '01/01/2015', @FECHAFIN = '01/01/2016';


-- (B)
-- CONTAR LA CANTIDAD DE ARTÍCULOS
EXECUTE CANTIDAD_ART_CLI @TIPOCONSULTA = 'ARTICULOS';

-- CONTAR LA CANTIDAD DE CLIENTES
EXECUTE CANTIDAD_ART_CLI @TIPOCONSULTA = 'CLIENTES';


-- (C)
-- UN EJEMPLO DE EJECUCIÓN
EXEC INS_VENDEDOR
    @NOM_VENDEDOR = 'NOMBRE',
    @APE_VENDEDOR = 'APELLIDO',
    @CALLE = 'CALLE ',
    @ALTURA = 123,
    @COD_BARRIO = 1,
    @NRO_TEL = 1234567890,
    @EMAIL = 'NOMBRE.APELLIDO@EXAMPLE.COM',
    @FEC_NAC = '01/01/2000';

-- BUSCO EL VENDEDOR INSERTADO CON MIS VALORES
-- USTEDES BUSQUEN SON VALORES INSERTADOS.
SELECT * FROM VENDEDORES WHERE NOM_VENDEDOR = 'NOMBRE';

-- (D)
-- UN EJEMPLO DE EJECUCIÓN
EXEC UPD_VENDEDOR
    @COD_VENDEDOR = 1,
    @NOM_VENDEDOR = 'NOMBRE1',
    @APE_VENDEDOR = 'APELLIDO1',
    @CALLE = 'CALLE EJEMPLO',
    @ALTURA = 456,
    @COD_BARRIO = 2,
    @NRO_TEL = 9876543210,
    @EMAIL = 'NOMBRE1.APELLIDO1@EXAMPLE.COM',
    @FEC_NAC = '10/05/1995';

-- VERIFICO LA MODIFICACIÓN EJEMPLO
SELECT * FROM VENDEDORES WHERE NOM_VENDEDOR = 'NOMBRE1';

-- (E)
-- EN EL CASO DE “MIS DATOS EN MI BASE” EL COD_VENDEDOR INSERTADO ES 10.

EXEC DEL_VENDEDOR @cod_vendedor = 10;
```

3.4

```
-- (A)
DROP PROCEDURE DETALLE_VENTAS;

-- (B)
DROP PROCEDURE CANTIDAD_ART_CLI;

-- (C)
DROP PROCEDURE INS_VENDEDOR;

-- (D)
DROP PROCEDURE UPD_VENDEDOR;

-- (E)
DROP PROCEDURE DEL_VENDEDOR;
```

BIBLIOGRAFÍA

- Gorman K., Hirt A., Noderer D., Rowland-Jones J., Sirpal A., Ryan D. & Woody B (2019) Introducing Microsoft SQL Server 2019. Reliability, scalability, and security both on premises and in the cloud. Packt Publishing Ltd. Birmingham UK
- Microsoft. SQL Server 2016. Disponible en: https://www.microsoft.com/es-es/sql-server/sql-server-2016
- Opel, A. & Sheldon, R. (2010). Fundamentos de SQL. Madrid. Editorial Mc Graw Hill
- Varga S., Cherry D., D'Antoni J. (2016). Introducing Microsoft SQL Server 2016 Mission-Critical Applications, Deeper Insights, Hyperscale Cloud. Washington. Microsoft Press

Atribución-No Comercial-Sin Derivadas

Se permite descargar esta obra y compartirla, siempre y cuando no sea modificado y/o alterado su contenido, ni se comercialice. Referenciarlo de la siguiente manera: Universidad Tecnológica Nacional Facultad Regional Córdoba (S/D). Material para la Tecnicatura Universitaria en Programación, modalidad virtual, Córdoba, Argentina.